Report for the Final Project for the Multimedia Technology Course

Dimoulas Ioannis, 03120083

The MediaLab.Assistant application was developed using Eclipse IDE for Java Developers – 2024-12. The Maven framework was used with Java JDK-21. After downloading and setting up the above prerequisites, the user is prompted to open a terminal in the application's root directory where pom.xml file is located and run the command *mvn clean install*. Finally, within Eclipse IDE, run the application by selecting Run > Run As > App.

The defined classes were organized into the following packages, located in the directory *Assistant/src/main/java*:

- **MediaLab.Assistant**: Contains the classes of the interface Controllers, as well as the *App.java* class, which contains the *main* method.
- **Models**: Includes the classes of the basic objects handled by the application, such as tasks, categories, etc.
- **Services**: Includes classes that manage the lists of Models' classes' objects defined throughout the application. They follow the Singleton Design Pattern to ensure proper access to shared objects and data consistency.
- **Utils**: Contains miscellaneous classes, including the Encoder and the Decoder, which handle the encoding and decoding, respectively, of the JSON files' data for storage purposes.

These JSON files are located in the medialab folder in the root directory and have the following structure:

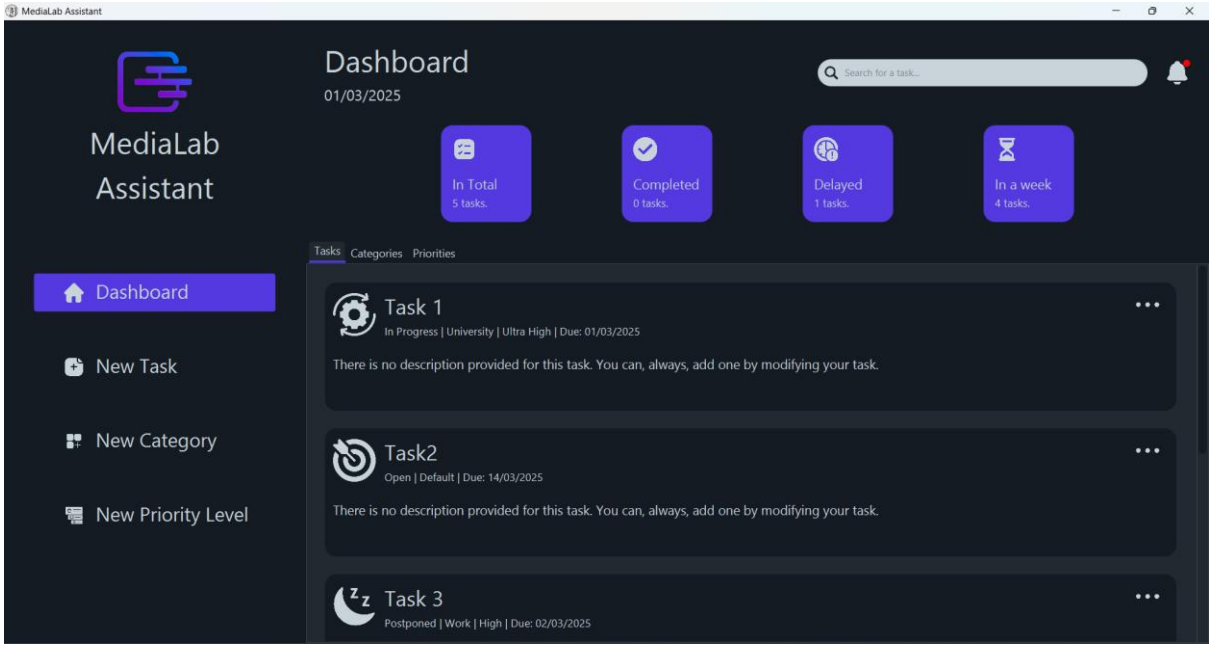
categories.json: [{"title": String, "description": String}, {…}]
priorities.json: [{"index": int, "title": String, "description": String}, {…}]
tasks.json: [{"title": String, "description": String, "deadline": String, "category": String, "priority": String, "Status": String, "notifications": [{"type": String, "notificationDate": String}, {…}]}, {…}]
notifications.json: [{"task": String, "type": String, "notificationDate": String}, {…}]

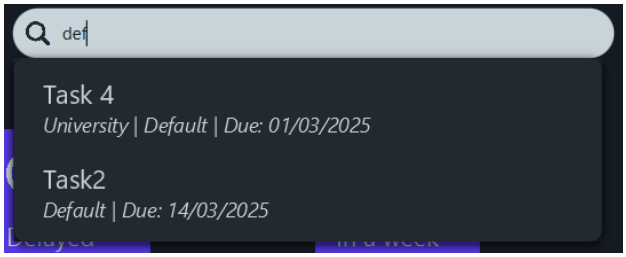
The name of the key is enclosed in quotation marks, followed by the expected value type. The {…} indicates the possible existence of similar objects within the list.

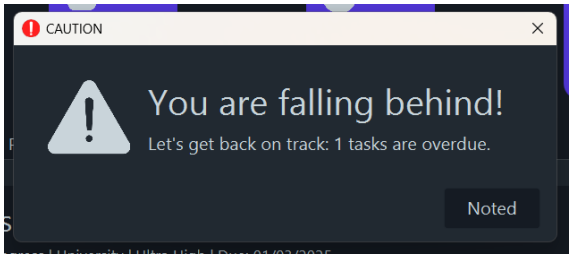
During the decryption process, the objects for categories and priorities are created first. Next, the tasks are defined. At this point, I should mention that I have assumed that the title of a task, a category, or a priority serves as its ID. In other words, there cannot be two distinct

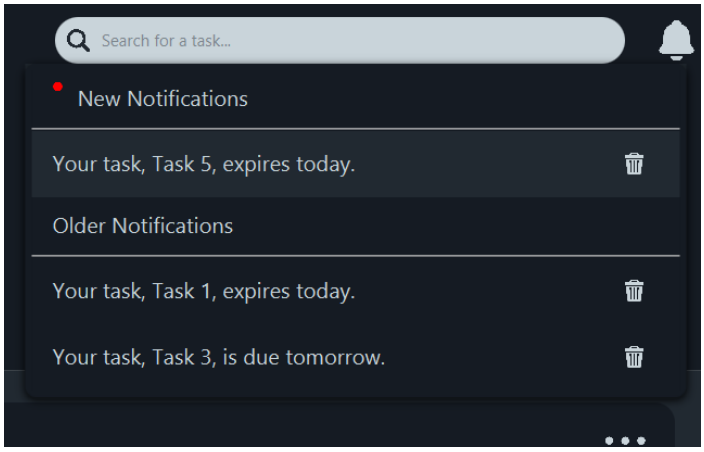
objects of these classes with the same title. Thus, the corresponding titles are recorded in the "category" and "priority" fields of *tasks.json*, and a search is performed in the list of the already defined objects to find the appropriate one. This ensures data consistency. As for *notifications.json*, the existence of this file is justified by my decision to maintain a history, which will include both read and unread notifications. Consequently, *tasks.json* contains the notifications set for each task; while *notifications.json* contains notifications whose display date has passed and have been viewed by the user. At the end of the decryption process, a scan is performed of the selected notifications for all tasks to identify any notifications whose display date has passed but which have not yet been viewed by the user.

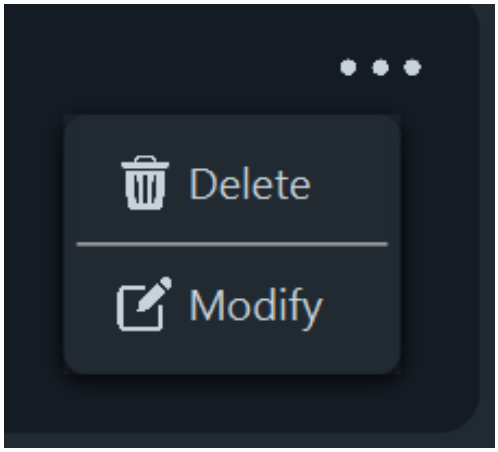
When the interface is opened, the application takes the user to the main screen, the Dashboard, which includes, among other things, a search bar for searching for tasks according to the assignment instructions, a bell icon for opening the notifications history, four information panels according to the specifications of the assignment, a menu on the left for navigating to different pages/functions, and a tab pane for viewing the defined tasks, categories, and priorities. Additionally, if there are overdue tasks, the appropriate popup screen appears when the application starts.

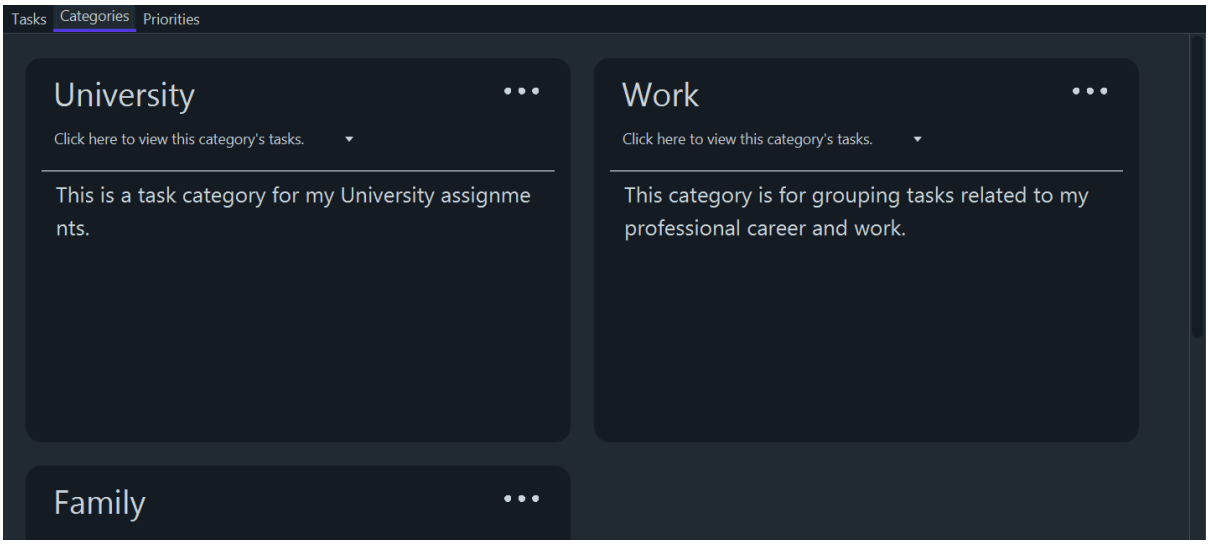


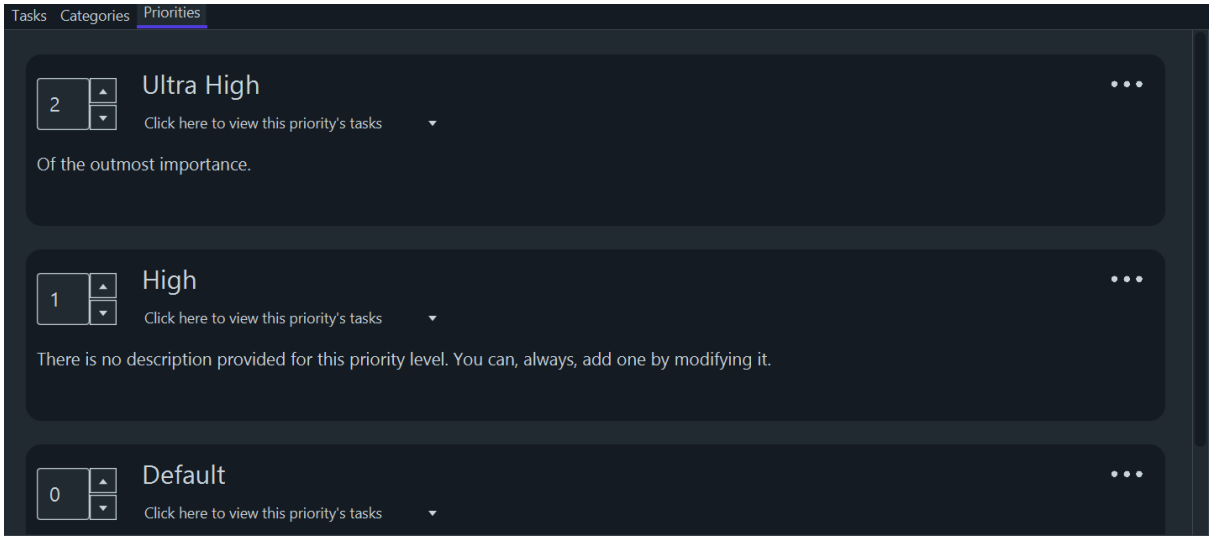






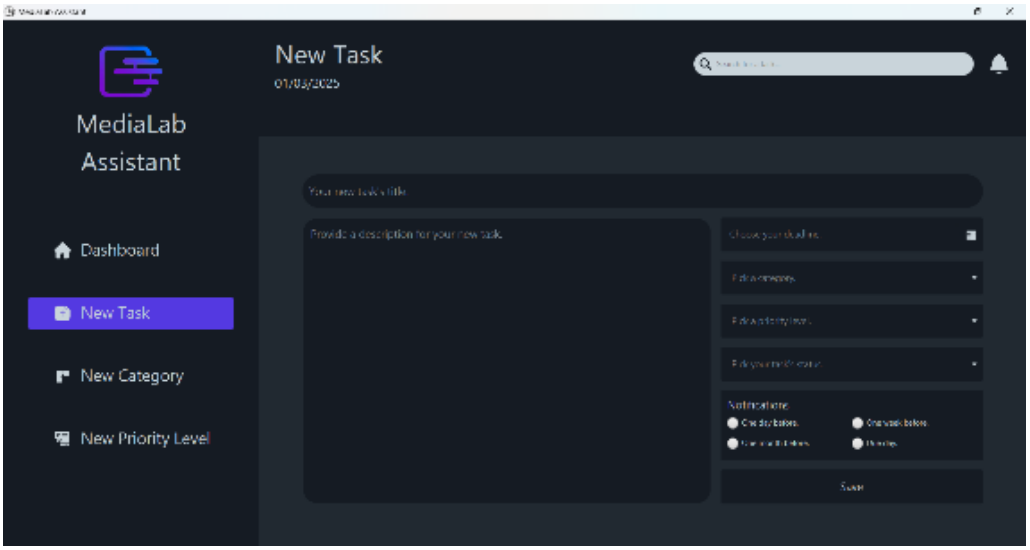


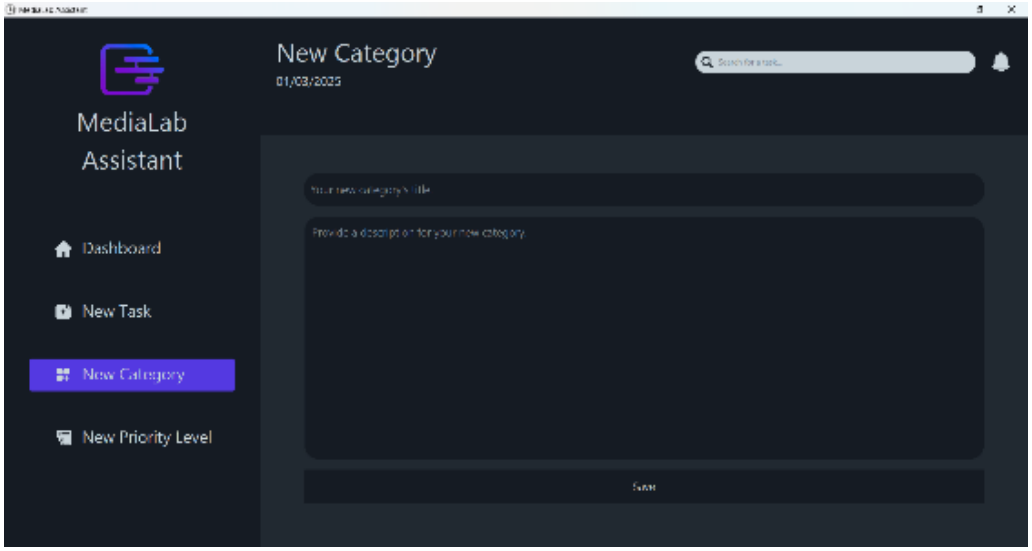


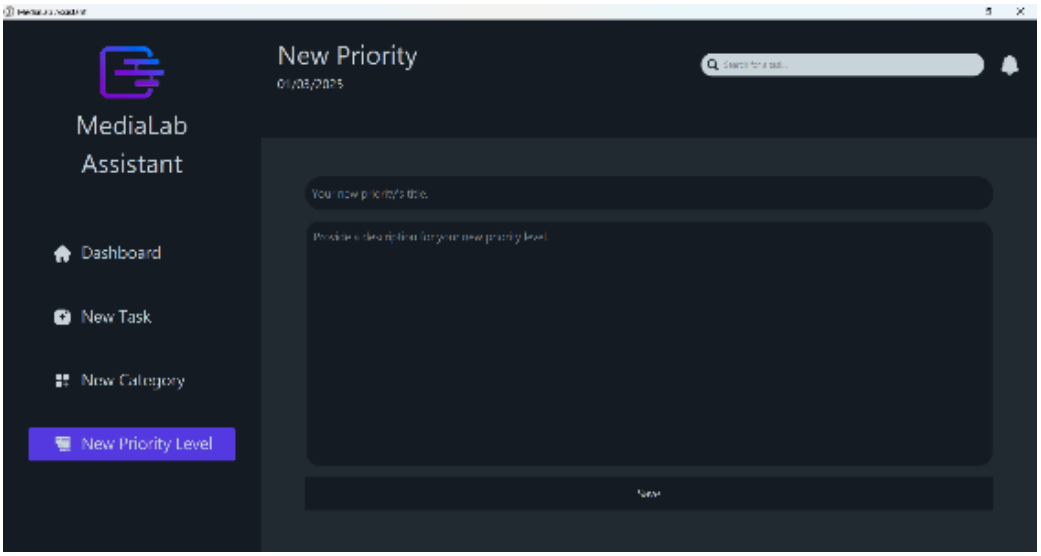


As shown in the images, the user can delete and edit notifications, tasks, categories, and priorities. For the categories, I added a description that wasn't necessary; the same goes for the priorities—to which I also added a priority level. The blue boxes are interactive, meaning the user can click on them so that only the relevant task cards appear in the tasks tab. Finally, if the user clicks on a card or, additionally, on the notifications and search options for tasks,

they are directed to the appropriate editing page for that item. Below, you can see the interfaces for creating new items.







Before creating or modifying an object, the appropriate validation is performed in accordance with our assumptions, and in appropriate cases, the corresponding error popup message is displayed.

In general, all specifications of the requirements were implemented, and the assumptions mentioned earlier were taken into account. The interfaces were designed using Scene Builder, and CSS files were used for styling. I also attempted to bind the various UI elements to the screen size; however, screen sizes other than my own are not optimized.

The class I chose to document according to the Javadoc tool specifications is Category.java in the Models package.